

Projektni zadatak

maj 2026.

Napraviti distribuirani sistem za automatsku obradu faktura. Sistem se sastoji od nezavisnih Java servisa koji komuniciraju putem mrežnih protokola, a svaki od njih predstavlja jednu fazu obrade faktura. Sve faze se izvršavaju sekvencijalno. Svaki servis mora imati detaljno logovanje svih akcija tako da se rad sistema može pratiti u realnom vremenu.

Faza 1 - Watcher servis

Ovaj servis prati promjene u folderu */invoices/inbox*. Kada se detektuje novi XML fajl, servis izvršava sljedeće akcije:

- Čita fajl i dobija sadržaj kao niz bajtova, a nakon dobijanja odgovora (ACK) od Parser servisa, fajl se premješta u folder */invoices/processed/*.
- Izračunava SHA-256 hash sadržaja fajla koji se šalje zajedno sa pročitanim bajtovima.
- Šalje sadržaj putem TCP socketa prema *Parser Servisu*. Poruka se sastoji od:
 - *Header* (4 bajta): *Big-endian integer* koji označava dužinu isključivo XML sadržaja u bajtovima (vrijednost N).
 - *Payload* (N bajtova): Pročitani bajtovi XML fajla.
 - *Footer* (32 bajta): Izračunati SHA-256 hash sadržaja fajla.

Voditi računa da se odjednom u folderu može kreirati na stotine fajlova i voditi računa da prilikom detektovanja promjena servis ne izgubi neke podatke i ne preoptereti dalje servise u lancu obrade. Može se implementirati rješenje sa nitima ili nekim *buffer*-om ili slično.

Faza 2 - Parser servis

Ovaj servis implementirati kao *Socket* server koji prihvata konekcije *Watcher servisa* i radi sljedeće:

- Prima bajtove i hash od *Watcher servisa*.
- Vraća ACK poruku sa nazivom fajla.
- Provjerava integritet: izračunava SHA-256 nad primljenim bajtovima i poredi sa primljenim *hash*-om. Ako se ne poklapaju odbacuje fakturu i loguje grešku.
- Parsira XML iz *byte[]*.
- Izvlači vrijednost *<id>* iz XML-a i provjerava duplikat u *Redis*-u. Ako ključ već postoji faktura je duplikat, odbacuje se i loguje.
- Ako nije duplikat: poziva *Validator Service* putem RMI.
- Ako je validacija uspješna: serijalizuje fakturu u JSON i stavlja na MQ topic *invoices.validated*.
- Ako validacija nije uspješna: stavlja na MQ topic *invoices.rejected* sa listom grešaka.

Faza 3 - Validator servis

Ovaj servis je zadužen za provjeru tačnosti podataka na fakturi. Implementirati kao RMI.

ValidationResult validate(Invoice invoice) throws RemoteException

Servis izvršava sljedeće validacije:

- JIB: tačno 13 cifara.
- Datum: ne smije biti u budućnosti, ne smije biti stariji od 365 dana.
- Stavke: mora biti barem jedna, quantity i unitPrice moraju biti pozitivni.
- Valuta: BAM, EUR, USD, CHF, GBP.
- Ukupan iznos (sum of quantity × unitPrice): mora biti > 0 i < 1.000.000.
- Provjera email formata.

ValidationResult sadrži: *boolean valid*, *List<String> errors*, *String validatorTimestamp*.

Faza 4 - Enrichment servis

Konsumira poruke sa MQ topica *invoices.validated*. Za svaku fakturu:

- Dohvata kurs valute prema BAM putem REST API-ja (npr. <https://open.er-api.com/v6/latest/BAM>).
- Prije poziva REST API-ja provjerava Redis *cache (rate:{currency}:{date})*. Ako postoji i nije stariji od 1 sata — koristi keširanu vrijednost i ne poziva API.
- Izračunava *totalBAM*, dodaje polja *totalBAM* i *rateSource* (vrijednost: CACHE ili API) u JSON fakturu.
- Ako REST API nije dostupan i nema *cache* — stavlja fakturu na MQ topic *invoices.retry* s brojačem pokušaja (*retryCount*).
- Stavlja obogaćenu fakturu na MQ topic *invoices.enriched*.

Faza 4b - Retry mehanizam

U *Enrichment servisu* implementirati poseban *listener* koji konzumira *invoices.retry* topic. Za svaku poruku:

- Čeka fiksno N sekundi (konfigurabilno, npr. 30s). Preporuka: korištenje *Delayed Queues* (ako broker podržava) ili asinhronog *ScheduledExecutorService*-a.
- Pokušava ponovo dohvatiti kurs.
- Ako uspije — nastavlja normalan tok (stavlja na *invoices.enriched*).
- Ako ne uspije i *retryCount* >= 3 — stavlja na *invoices.failed* s razlogom RATE_UNAVAILABLE.
- Ako ne uspije i *retryCount* < 3 — vraća na *invoices.retry* s uvećanim brojačem.

Faza 5 - Approval servis

Konsumira poruke s MQ topica *invoices.enriched*. Za svaku fakturu:

- Izračunava ukupan iznos u BAM (*totalBAM*).
- Ako je *totalBAM* <= 100 — automatski odobrava fakturu i stavlja na topic *invoices.approved*.
- Ako je *totalBAM* > 100 — stavlja fakturu na topic *invoices.pending_approval* gdje se čeka odobrenje.

Approval Service ne nastavlja obradu dok faktura nije odobrena ili odbijena od strane *Approval klijenta*. Ovaj klijent je GUI aplikacija koja konstantno treba da prati postoje li novi zahtjevi za potvrdu faktura. Kada novi zahtjev stigne, prikazuju se svi podaci sa fakture (formatirano) i korisnik ima opcije da odobri ili odbije fakturu. U slučaju odbijanja, obavezno unosi razlog u tekstualno polje. Nakon odlučivanja fakturu prebaciti u *invoices.approved* ili *invoices.rejected*.

Faza 6 - Agregator servis

Konzumira poruke sa svih *topic*-a i ažurira statuse u Redis-u. Za svaku fakturu:

- Čuva kompletan JSON zapis fakture u Redis (*invoice:{id}* → JSON string).

Za fakture koje su na *invoices.approved* *topic*-u:

- Ažurira Redis *hash* za klijenta (*client:{jib}* → polja: *totalBAM*, *count*, *lastInvoiceId*, *lastUpdated*).
- Šalje UDP multicast notifikaciju na grupu 239.0.0.1:4446, format JSON: *{invoiceId, clientName, totalBAM, status, timestamp}*.

Monitor klijent

Može se pokrenuti kao N nezavisnih instanci. Svaka instanca:

- Pridružuje se UDP multicast grupi 239.0.0.1:4446.
- Prima i parsira JSON notifikacije.
- Prikazuje GUI tabelu sa aktuelnim podacima.

Nije dozvoljeno koristiti REST API polling umjesto *multicast*.

REST API Service

HTTP server koji čita isključivo iz Redis-a. Endpointi (svi odgovori JSON):

- GET /api/invoices — lista svih obrađenih faktura
- GET /api/invoices/{id} — kompletna faktura
- GET /api/invoices?status={status} — filtriranje (*approved* / *rejected* / *failed* / *pending*)
- GET /api/clients/{jib}/summary — ukupno BAM, broj faktura, zadnja faktura
- GET /api/stats — ukupno faktura, ukupno BAM, prosječan iznos, broj pending

Primjer fakture:

```
<?xml version="1.0" encoding="UTF-8"?>
<invoice>
  <id>INV-2025-001</id>
  <client>
    <name>ETF</name>
    <jib>1234567891012</jib>
    <email>fakture@etf.com</email>
  </client>
  <date>2025-05-01</date>
  <currency>EUR</currency>
  <items>
    <item>
      <description>Web development</description>
```

```
<quantity>10</quantity>  
<unitPrice>50.00</unitPrice>  
</item>  
<item>  
  <description>Hosting</description>  
  <quantity>1</quantity>  
  <unitPrice>120.00</unitPrice>  
</item>  
</items>  
</invoice>
```

Za odbranu pripremiti minimalno 100 različitih faktura.

Sve što nije precizno definisano, studenti mogu implementirati na način koji smatraju najboljim.